The Design and Formal Analysis of the Quantum Message Authentication Code

John G. Underhill

Quantum Resistant Cryptographic Solutions Corporation, Canada

Abstract. QMAC is a message authentication code that combines a polynomial hash over the field $GF(2^{256})$ with keys derived from the cSHAKE customizable hash function. This paper presents a formal analysis of the construction and provides a detailed examination of its algebraic structure, security properties, and implementation alignment. The analysis includes an engineering description of the mechanism based directly on the reference source code, a model-based treatment of the polynomial hash function and its ε almost universal bounds, and a reduction based proof of existential unforgeability under chosen message attack. The reduction separates the universal hashing component from the pseudo-randomness of the cSHAKE based key derivation and yields explicit bounds that depend on adversarial query limits and field size. Because QMAC derives both the hash key and the finalization mask from a single per-message nonce, the construction is analysed as a *nonce based* MAC; we make the nonce-uniqueness requirement explicit, prove unforgeability for nonce respecting adversaries, and document the algebraic key-recovery attack that applies if a nonce is ever reused under a fixed key. Quantum security considerations are examined within a quantum oracle model and are related to known results on polynomial hashing and cSHAKE in the presence of quantum adversaries. The paper concludes with a comparative and implementation oriented evaluation of QMAC and a discussion of practical deployment conditions and limitations.

E-mail: contact@qrcscorp.ca (John G. Underhill)

Contents

1 Introduction **4**
1.1 Background and Motivation 4
1.2 Contributions 4
1.3 Organization of the Paper 5

2 Engineering Description of QMAC **5**
2.1 Parameters, Constants, and State Layout 5
2.2 Key Derivation with cSHAKE 6
2.3 Message Encoding, Padding, and Block Processing 6
2.4 Polynomial Multiplication over $GF(2^{256})$ 7
2.5 Tag Finalization and Output 7
2.6 Auxiliary Operations 7

3 Mathematical Preliminaries **8**
3.1 Notation 8
3.2 The Field $GF(2^{256})$ and the Irreducible Polynomial 8
3.3 Carryless Multiplication and Reduction 9
3.4 Polynomial Hashing and Universal Hash Functions 9

4 The QMAC Construction **10**
4.1 High Level Construction 10
4.2 Relation to GMAC, Poly1305, and KMAC 11
4.3 Domain Separation and Customization Strings 11

5 Security Definitions **12**
5.1 Adversarial Model and Oracles 12
5.2 MAC Security Experiment and EUF-CMA 12
5.3 Universal Hashing and Collision Bounds 13
5.4 PRF Security of cSHAKE Based Key Derivation 13

6 Algebraic Analysis of the Polynomial Hash **14**
6.1 Collision Properties over $GF(2^{256})$ 14
6.2 Epsilon Almost Universal Bounds 15
6.3 Epsilon Almost Xor Universal Properties 16
6.4 Justification of the Modulus Choice 17

7 Security Proofs for QMAC **17**
7.1 Universality Lemma for QMAC Polynomial Hashing 17
7.2 Idealized Construction and One-Time Analysis 18
7.3 Reduction from EUF-CMA to cSHAKE PRF Security 19
7.4 Combined Security Bound 20
7.5 Discussion of Tightness and Parameter Choices 20
7.6 Security under Nonce Reuse 21
7.7 Quantum Adversaries and Oracle Models 22
7.8 Quantum Bounds for Polynomial MACs 22
7.9 Quantum Security of cSHAKE Key Derivation 23
7.10 Resulting Quantum Security Levels 23

8 Cryptanalytic Analysis and Comparative Evaluation **24**
8.1 Diffusion and Linear Mixing Properties 24
8.2 Comparison with GMAC, Poly1305, and KMAC 25
8.3 Work Factor Estimates and Usage Guidelines 25

9 Implementation Conformance and Side Channel Considerations 26
9.1 Mapping Between Model and Reference Implementation 26
9.2 Constant Time Behavior and Microarchitectural Issues 27
9.3 Randomness and Nonce Handling 28
9.4 Key and State Erasure 28

10 Conclusion 29
10.1 Summary of Results 29
10.2 Limitations and Future Work 30
10.3 Implications for Deployment 30

11 References 31

1 Introduction

Message authentication codes are essential components of modern cryptographic protocols. They provide integrity and authenticity guarantees for messages that travel across potentially adversarial networks. Widely deployed polynomial authenticators such as GMAC and Poly1305 were standardized around classical security targets and compact tags. GMAC evaluates a polynomial over $GF(2^{128})$, while Poly1305 evaluates a polynomial modulo $2^{130} - 5$. These constructions remain efficient and widely deployed, but their smaller authentication domains, algebraic structure, nonce-misuse fragility, and quantum-oracle considerations motivate larger-field alternatives for high-assurance post-quantum symmetric deployments.

QMAC is a message authentication code that combines a polynomial hash over the field $GF(2^{256})$ with keys derived from the customizable hash function cSHAKE. Its structure is closest to polynomial MACs such as GMAC and Poly1305: authentication is obtained by evaluating an algebraic hash and applying a per-message mask. QMAC replaces the smaller polynomial domains and compact tags of those constructions with a 256-bit binary field, a 256-bit tag, and cSHAKE based derivation of a fresh per-nonce hash key and finalization mask. The polynomial hash enables fast evaluation on common architectures, and the use of cSHAKE provides a flexible key-derivation interface and strong pseudo-randomness guarantees under the stated model.

This paper presents a formal cryptanalysis of QMAC, addressing both its algebraic structure and its security properties. An implementation-aligned engineering level description of the construction is provided to establish a precise link between the reference implementation and the mathematical model used in the analysis. The algebraic properties of the polynomial hash are examined in detail, including proofs of its ε almost universal behavior over an irreducible polynomial field of degree 256. These properties are combined with assumptions on the pseudo-randomness of the cSHAKE based key derivation to obtain explicit unforgeability bounds in the chosen message setting. The security model separates the universal hashing component from the key derivation component, and the resulting reduction yields interpretable expressions for the adversarial advantage.

The analysis also considers attackers with quantum capabilities. The polynomial hash is evaluated in the context of quantum oracle access, and the use of cSHAKE is examined under standard assumptions about the behavior of extendable output functions in the presence of quantum queries. These results are related to work on polynomial MACs and hash based constructions in the quantum setting.

1.1 Background and Motivation

The design of QMAC is motivated by several goals. The first is to obtain a MAC that is suitable for symmetric systems targeting post-quantum security margins without depending on asymmetric primitives. The second is to retain the simplicity and efficiency of polynomial authentication while increasing the field size and tag length beyond the 128-bit regimes used by GMAC and Poly1305. The third is to provide a mechanism that integrates cleanly with cSHAKE, which is already used in modern post-quantum protocol designs. By using a polynomial hash over a larger field, QMAC retains a compact and regular structure that maps well to vectorized instructions on common CPUs.

1.2 Contributions

This paper provides the following contributions.

- An implementation-aligned engineering level description of QMAC based directly on the reference implementation. This description captures the exact operational

details of the construction, including block processing, field arithmetic, reduction rules, cSHAKE based key derivation, and state management.

- A formal model of the polynomial hash over $GF(2^{256})$, including an injective message-encoding (the 10^* padding together with an appended length block) and proofs of its ε almost universal collision bounds.
- An explicit reduction based proof of existential unforgeability under chosen message attack in the *nonce based* setting. The reduction separates the universal hashing component from the pseudo-randomness of the cSHAKE based key derivation and yields a concrete bound involving both contributions. We also state and analyse the algebraic key-recovery attack that arises under nonce reuse, making precise the conditions under which the proof applies.
- A discussion of quantum security in a quantum oracle model, including the behavior of polynomial hashing and cSHAKE under quantum queries.
- A comparative cryptanalytic evaluation of QMAC in relation to GMAC and Poly1305, highlighting algebraic properties, tag-size considerations, nonce-misuse behavior, quantum-model considerations, and deployment characteristics.

1.3 Organization of the Paper

Section 2 provides an engineering level description of QMAC that reflects the exact behavior of the reference implementation. Section 3 introduces the mathematical preliminaries, including notation, the field $GF(2^{256})$, and the structure of the polynomial hash. Section 4 gives a formal specification of QMAC in functional terms. Section 5 defines the security notions and adversarial model. Section 6 presents the algebraic analysis of the polynomial hash and its universality bounds. Section 7 contains the security proofs, including the reduction for existential unforgeability and the Q1/Q2 quantum discussion. Section 8 provides the comparative cryptanalytic evaluation. Section 9 discusses implementation conformance and side channel issues. The paper concludes with a summary of results, limitations, and directions for future work.

2 Engineering Description of QMAC

This section provides a functional description of QMAC based on the reference C implementation. All operations are described at the level of mathematical data flow, independent of optimization details. The purpose is to capture the exact behavior of the algorithm as it is defined by the code, expressed in a form suitable for formal reasoning.

2.1 Parameters, Constants, and State Layout

QMAC operates on fixed size blocks and uses fixed size subkeys and internal state. These parameters are expressed in the implementation as constants and are formalized as follows.

$$\begin{aligned} \text{Block size} &= 32 \text{ bytes}, \\ \text{Key size} &= 32 \text{ bytes}, \\ \text{Tag size} &= 32 \text{ bytes}, \\ \text{Nonce size} &= 32 \text{ bytes}. \end{aligned}$$

The internal state consists of three field elements in $GF(2^{256})$, each represented as a 256 bit vector:

$$H \in GF(2^{256}), \qquad F \in GF(2^{256}), \qquad Y \in GF(2^{256}).$$

During initialization Y is set to the zero element.

2.2 Key Derivation with cSHAKE

Initialization derives the pair of subkeys (H, F) using a single call to cSHAKE that absorbs a secret key K, a nonce N, and an optional information string info. In the reference code (`qsc_qmac_initialize`) the secret key is supplied as the keyed input of cSHAKE, while the nonce and the information string are supplied as the cSHAKE function-name and customization inputs respectively, providing per-invocation domain separation. The implementation then squeezes one output block and uses the first 64 bytes, taking the first 32 bytes as H and the next 32 bytes as F. Mathematically, we express this as:

$$(H, F) = \mathrm{Split}_{256,256}\big(\mathrm{cSHAKE}(K, N, \mathrm{info})\big).$$

The function $\mathrm{Split}_{256,256}$ takes the first 256 bits of the cSHAKE output as H and the next 256 bits as F.

Formally:

$$H = \mathrm{cSHAKE}(K, N, \mathrm{info})[0..255], \qquad F = \mathrm{cSHAKE}(K, N, \mathrm{info})[256..511].$$

The accumulator is initialized as:

$$Y = 0_{256}.$$

No additional key material is retained after initialization.

2.3 Message Encoding, Padding, and Block Processing

A message M of $b = |M|$ bytes is divided into blocks of 32 bytes. Let

$$M = M_1 \parallel M_2 \parallel \cdots \parallel M_{m-1} \parallel \widetilde{M}_m, \qquad |M_i| = 32 \text{ bytes for } i < m,$$

where $\widetilde{M}_m$ is the (possibly empty) trailing partial block. The reference code does not zero-pad the trailing block silently. Instead it applies an unambiguous 10^* padding followed by a dedicated length block, matching the behaviour of `qmac_compute_final`:

- A single byte `0x80` is appended after $\widetilde{M}_m$, and the remainder of the block is filled with zero bytes to reach 32 bytes. If the message length is an exact multiple of 32 (including the case $\widetilde{M}_m$ empty), a fresh block `0x80` $\texttt{0x00}^{31}$ is produced. This yields exactly $m = \lfloor b/32 \rfloor + 1$ encoded message blocks $M_1, \ldots, M_m$.
- A final *length block* L is then appended. It encodes the processed message length b in bytes as a little-endian 64-bit integer in its first eight bytes, with the remaining bytes zero except for the most significant byte, which is set to `0x01` as a domain-separation marker. In the code this is the buffer `lbuf` formed by `qsc_intutils_le64to8` followed by `lbuf[31] = 0x01`.

The full sequence of blocks that is absorbed into the accumulator is therefore

$$M_1, M_2, \ldots, M_m, L, \qquad \ell := m + 1 = \lfloor b/32 \rfloor + 2,$$

where ℓ denotes the total number of $GF(2^{256})$ blocks processed. The public update interface accepts nonzero message fragments; an empty message is represented at the algorithmic level by initialization followed directly by finalization, which still absorbs the padding block and length block. The formal message space therefore includes the empty string, while the public update call itself requires nonzero input. Each block is interpreted

as an element of $GF(2^{256})$ through a fixed encoding map (the implementation fixes a little-endian word ordering, copying 32 bytes into four 64-bit words):

$$X_i = \text{Encode}(\text{block}_i) \in GF(2^{256}), \qquad 1 \le i \le \ell.$$

The map Encode is a fixed $GF(2)$-linear bijection between 32-byte strings and field elements; only this property is used in the analysis, so the specific byte order is immaterial to the security bounds. For each block the accumulator is updated by:

$$Y \leftarrow H \cdot (Y \oplus X_i),$$

where multiplication and addition are taken in $GF(2^{256})$. This update rule is implemented exactly by `qmac_block_update`, which first xors the block into Y and then multiplies by H.

2.4 Polynomial Multiplication over $GF(2^{256})$

The field $GF(2^{256})$ is instantiated as the quotient ring:

$$GF(2)[x]/\langle m(x)\rangle,$$

where the C implementation uses the irreducible pentanomial

$$m(x) = x^{256} + x^{10} + x^5 + x^2 + 1.$$

Every field element is represented as a 256 bit vector corresponding to a polynomial of degree at most 255.

Given two field elements A and B, QMAC computes:

$$A \cdot B = (A(x)B(x)) \bmod m(x).$$

The reduction is equivalent to repeatedly replacing terms x^{256+k} with:

$$x^k \oplus x^{k+2} \oplus x^{k+5} \oplus x^{k+10}, \qquad 0 \le k < 256.$$

This reduction rule captures exactly the bit level behavior of the implementation while remaining agnostic to the carryless multiply method used in the C code.

2.5 Tag Finalization and Output

After the last message block M_m and the length block L have been absorbed (so that the accumulator holds Y_ℓ with $\ell = m + 1$), the accumulator is combined with the finalization key F:

$$T = Y_\ell \oplus F.$$

The tag T is then returned as a 32 byte value obtained by applying the inverse of the encoding map. The `0x80` marker and the appended length block are part of the hashed input; finalization itself performs no further mixing beyond the field operations already described and the final xor with F.

State erasure is performed in the implementation, but this does not affect the mathematical semantics of the construction.

2.6 Auxiliary Operations

The C implementation includes a set of utility functions for integer conversion, zero padding, and secure memory clearing. In the formal description these correspond to the following abstract operations:

- A fixed $GF(2)$-linear bijection Encode from 32 byte strings to $GF(2^{256})$.
- An injective padding map that applies $\texttt{10}^*$ padding to the trailing block and appends a length block encoding $|M|$, as defined above.
- Erasure of temporary values after use.

None of these affect the logical structure of QMAC and are treated as standard functional primitives.

This engineering description defines QMAC as a composition of cSHAKE based key derivation and a polynomial hash over $GF(2^{256})$, expressed in a form that can be analyzed independently of implementation details but remains fully aligned with the behavior of the reference code.

3 Mathematical Preliminaries

This section introduces the notation, algebraic structures, and hashing framework used in the analysis of QMAC. These definitions abstract the implementation level behavior of the algorithm into formal mathematical operations that will support the subsequent security proofs.

3.1 Notation

We use the following notation throughout the paper.

- $\{0,1\}^n$ denotes the set of bit strings of length n.
- For bit strings A and B, the symbol $A \parallel B$ denotes concatenation.
- For a string X, the notation $X[i..j]$ refers to the substring spanning bit positions i through j inclusive, under a fixed ordering convention. The analysis depends only on the encoding map being a fixed bijection, not on a particular endianness.
- Random sampling from a finite set S is written as $x \xleftarrow{\$} S$.
- For a probabilistic algorithm $\mathcal{A}$, $\Pr[\mathcal{A} \Rightarrow 1]$ denotes the probability that $\mathcal{A}$ outputs 1.
- We write $\mathsf{negl}(\lambda)$ for a function negligible in the security parameter λ.
- All polynomial expressions are over $GF(2)$ unless otherwise stated.

A 256 bit string corresponds naturally to a polynomial in $GF(2)[x]$ of degree at most 255. This correspondence is used to define the field arithmetic in QMAC.

3.2 The Field $GF(2^{256})$ and the Irreducible Polynomial

QMAC operates over the finite field $GF(2^{256})$, defined as the quotient ring

$$GF(2^{256}) = GF(2)[x]/\langle m(x)\rangle,$$

where the modulus $m(x)$ is the irreducible pentanomial

$$m(x) = x^{256} + x^{10} + x^5 + x^2 + 1.$$

This polynomial is irreducible over $GF(2)$ and provides an efficient basis for polynomial reduction. Every element of $GF(2^{256})$ is represented as a polynomial

$$A(x) = a_{255}x^{255} + a_{254}x^{254} + \cdots + a_1x + a_0,$$

with coefficients $a_i \in GF(2)$. The implementation fixes an encoding of such polynomials as 256 bit strings using a little-endian word ordering (four 64-bit words, least significant word first). Writing the encoding abstractly, Encode is the fixed $GF(2)$-linear bijection

$$A \in \{0,1\}^{256} \quad \longleftrightarrow \quad A(x) = \sum_{i=0}^{255} a_i \, x^i,$$

where the coefficient vector $(a_0, \ldots, a_{255})$ is obtained from the byte string by the implementation's fixed word and bit ordering. Field addition is bitwise xor of the string representations. Multiplication is defined by polynomial multiplication modulo $m(x)$, described next. Only the bijectivity and $GF(2)$-linearity of Encode are used in the security proofs.

3.3 Carryless Multiplication and Reduction

Given $A, B \in GF(2^{256})$, their product in the field is defined as:

$$A \cdot B = \big(A(x)B(x)\big) \bmod m(x).$$

The polynomial product $A(x)B(x)$ has degree at most 510 and can be written as:

$$C(x) = \sum_{i=0}^{510} c_i x^i.$$

The reduction modulo $m(x)$ is performed by eliminating all terms of degree at least 256 using the relation:

$$x^{256} \equiv x^{10} \oplus x^5 \oplus x^2 \oplus 1 \pmod{m(x)}.$$

Thus, for any $k \geq 0$, the term x^{256+k} reduces to:

$$x^{256+k} \equiv x^k \oplus x^{k+2} \oplus x^{k+5} \oplus x^{k+10}.$$

The full field product is obtained by repeatedly applying this rule to eliminate all terms of degree 256 through 510, after which the remaining polynomial has degree at most 255 and represents the final field element.

This mathematical reduction corresponds exactly to the behavior of the reference implementation, which processes bits of the product from high to low and applies the same reduction map.

3.4 Polynomial Hashing and Universal Hash Functions

QMAC uses a polynomial hash evaluated in $GF(2^{256})$ with a secret key $H \in GF(2^{256})$. A message M is first mapped, by the injective padding of Section 2.3, to a sequence of field elements

$$\text{Encode-pad}(M) = (X_1, X_2, \ldots, X_\ell), \qquad \ell = \lfloor |M|/32 \rfloor + 2,$$

where $X_1, \ldots, X_{\ell-1}$ are the $\texttt{10}^*$-padded message blocks and X_ℓ is the length block. Throughout, ℓ denotes the number of processed blocks, which exceeds the raw message-block count by exactly the padding and length block. The QMAC polynomial hash is then:

$$Y_0 = 0_{256}, \qquad Y_i = H \cdot (Y_{i-1} \oplus X_i), \qquad 1 \leq i \leq \ell.$$

The value Y_ℓ is the internal hash state before finalization.

A family of hash functions $\mathcal{H} = \{h_H\}_{H \in \mathcal{K}}$ is called ε almost universal (AU) if for any two distinct messages M and M':

$$\Pr_{H \xleftarrow{\$} \mathcal{K}} [h_H(M) = h_H(M')] \leq \varepsilon.$$

It is called ε almost xor universal (AXU) if for all $M \neq M'$ and all Δ:

$$\Pr_{H \xleftarrow{\$} \mathcal{K}} [h_H(M) \oplus h_H(M') = \Delta] \leq \varepsilon.$$

Polynomial hashing over $GF(2^n)$ is well known to form an ε AU family with ε proportional to 2^{-n}. For QMAC, with $n = 256$, the collision probability is bounded by:

$$\varepsilon \leq \ell \cdot 2^{-256},$$

where ℓ is the number of message blocks. These bounds will be used in the subsequent security analysis.

4 The QMAC Construction

This section gives a formal and implementation agnostic description of QMAC. The construction presented here is derived directly from the behavior of the reference implementation but is expressed in mathematical terms suitable for analysis. All arithmetic takes place in the field $GF(2^{256})$ defined by the irreducible polynomial:
$m(x) = x^{256} + x^{10} + x^5 + x^2 + 1$.

4.1 High Level Construction

Let $\mathcal{K} = \{0,1\}^{256}$ be the key space, $\mathcal{N} = \{0,1\}^{256}$ the nonce space, $\mathcal{I}$ an arbitrary information space, and $\mathcal{M} = \{0,1\}^*$ the message space. The tag space is $\mathcal{T} = \{0,1\}^{256}$.

Initialization proceeds by deriving two field elements (H, F) from a master key $K \in \mathcal{K}$, a nonce $N \in \mathcal{N}$, and an auxiliary string info $\in \mathcal{I}$. The derivation uses a single cSHAKE call in which K is the keyed input and (N, info) supply the function-name and customization inputs, providing domain separation. The derivation is formalized as:

$$(H, F) = \text{Split}_{256,256}(\text{cSHAKE}_K(N, \text{info})),$$

where the first 256 output bits define H and the next 256 bits define F. A fresh nonce N is used for each message, so that each authenticated message is processed under an independently derived pair (H, F); this requirement is essential and is made precise in the security model of Section 5.

The message M is mapped to the block sequence

$$\text{Encode-pad}(M) = (X_1, X_2, \ldots, X_\ell), \qquad \ell = \lfloor |M|/32 \rfloor + 2,$$

where $X_1, \ldots, X_{\ell-1}$ are the $\texttt{10}^*$-padded message blocks and X_ℓ is the length block of Section 2.3, each interpreted as an element of $GF(2^{256})$ by the fixed bijection Encode.

The accumulator state is initialized as:

$$Y_0 = 0_{256}.$$

The polynomial hash is computed by the recurrence:

$$Y_i = H \cdot (Y_{i-1} \oplus X_i), \qquad 1 \leq i \leq \ell,$$

where multiplication and addition are the operations in $GF(2^{256})$.

Finalization produces the tag

$$T(M) = Y_\ell \oplus F.$$

Thus QMAC is a Carter Wegman style construction that combines a universal hash function with an independent pseudo-random value obtained from cSHAKE. The universal hashing properties of the polynomial map and the pseudo-randomness of the cSHAKE output form the basis for the security analysis.

4.2 Relation to GMAC, Poly1305, and KMAC

QMAC is structurally closest to polynomial authenticators such as GMAC and Poly1305. GMAC evaluates a polynomial over $GF(2^{128})$ and applies a block-cipher-derived mask. Poly1305 evaluates a polynomial modulo $2^{130} - 5$ and applies a one-time pad derived by the surrounding AEAD construction. QMAC uses the same broad Carter-Wegman pattern but evaluates its polynomial hash over $GF(2^{256})$ and derives both the hash key H and finalization mask F from cSHAKE using the master key, nonce, and application information string.

The larger field changes the universal-hashing term from the 128-bit regime to the 256-bit regime: for messages of at most ℓ processed blocks, the root-counting bound becomes $\ell \cdot 2^{-256}$ rather than a bound in a 128-bit field or near-128-bit modulus. The 256-bit tag also gives a larger generic tag-guessing margin than the compact tags used by GMAC and Poly1305. These changes do not make QMAC misuse resistant; as with polynomial MACs generally, reuse of a nonce-derived pair (H, F) exposes algebraic relations that lead to per-nonce forgery. The purpose of the larger field and cSHAKE derivation is to provide a more conservative nonce-respecting polynomial MAC, not to remove the nonce-uniqueness requirement.

KMAC has a different proof profile. It is a keyed sponge MAC analyzed directly through the PRF behavior of the Keccak sponge and does not expose an explicit polynomial hash key. QMAC instead separates the universal-hashing layer from the pseudo-random mask layer. This gives a transparent Carter-Wegman style reduction, but also means the construction is explicitly nonce based: each message must be authenticated under a unique nonce so that the derived pair (H, F) is not reused.

4.3 Domain Separation and Customization Strings

The security of QMAC relies on the independent derivation of H and F from the master key. In the reference implementation the master key K is the keyed input of cSHAKE, while the nonce N and the information string info are supplied as the cSHAKE function-name and customization inputs. The output is squeezed once and split, taking the first 256 bits as H and the next 256 bits as F.

Writing $\text{cSHAKE}_K[N, S]$ for cSHAKE keyed by K with function-name string N and customization string S, the derivation is

$$(H, F) = \text{Split}_{256,256}(\text{cSHAKE}_K[\,N,\ \text{info}\,])\,.$$

Two properties follow under the assumed pseudo-randomness of cSHAKE. First, for a fixed key the outputs on distinct nonces (and customizations) are independent and uniform, so distinct messages, which are processed under distinct nonces, receive independent pairs (H, F). Second, H and F extracted from a single output are themselves independent and uniform. The information string info supplies application-level domain separation so that a given master key is not shared across protocol components without distinguishing context.

This domain separation, together with the per-message nonce, is essential for the nonce-respecting security argument and will be referenced later in the cryptanalysis sections. It also prevents tags generated under distinct nonces from sharing the same polynomial hash key and mask. The quantum discussion in Section 7.7 treats known hidden-period attacks separately and does not claim a complete Q2 proof.

5 Security Definitions

The security of QMAC is analyzed in the standard framework for message authentication codes. This section defines the adversarial model, oracle access rules, and advantage functions used in the formal reductions. The definitions abstract the behavior of the implementation into standard security experiments.

5.1 Adversarial Model and Oracles

An adversary $\mathcal{A}$ is modeled as a probabilistic algorithm that interacts with a tagging oracle. QMAC derives *both* subkeys (H, F) from the nonce, so it is analysed as a *nonce based* MAC: the nonce is an explicit input to the oracle rather than a value fixed once at initialization. The oracle is defined with respect to a secret master key K and an information string info sampled during initialization.

The tagging oracle takes a nonce and a message and is defined as:

$$\mathsf{Tag}(N, M) = T_N(M) = Y_\ell \oplus F, \qquad (H, F) = \mathrm{KDF}(K, N, \mathrm{info}),$$

where Y_ℓ is computed using the polynomial hash of Section 3 over the padded encoding of M.

We require the adversary to be *nonce respecting*: the nonces $N_1, \ldots, N_q$ used in its q tagging queries are pairwise distinct. This reflects the implementation contract that a fresh nonce is used for every message under a fixed key. After interacting with the oracle, the adversary outputs a candidate forgery (N^*, M^*, T^*).

A forgery is valid if:

- the pair (N^*, M^*) was never the subject of a tagging query, i.e. there is no query index i with $(N_i, M_i) = (N^*, M^*)$, and
- $T^* = T_{N^*}(M^*)$ under the secret key.

The forgery nonce N^* *may* coincide with one of the queried nonces (this is the strongest case the adversary can attempt, and is analysed explicitly), but the queried nonces themselves must be distinct. Trivial forgeries are disallowed: the adversary may not simply replay a tag for a queried pair (N_i, M_i). The case of *nonce reuse in tagging queries*, which violates the nonce-respecting requirement, is not covered by the unforgeability theorem; the attack that becomes possible in that case is analysed separately in Section 7.6.

5.2 MAC Security Experiment and EUF-CMA

The unforgeability of QMAC is defined using the existential unforgeability under chosen message attack (EUF-CMA) experiment, adapted to the nonce based setting.

Experiment $\mathsf{EUF\text{-}CMA}^{\mathcal{A}}_{\mathrm{QMAC}}$ (nonce respecting):

1. Sample $K \xleftarrow{\$} \mathcal{K}$ and choose info as appropriate for the protocol.
2. Give $\mathcal{A}$ oracle access to $\mathsf{Tag}(N, M)$, where on its i-th query it submits (N_i, M_i); the experiment computes $(H_i, F_i) = \mathrm{KDF}(K, N_i, \mathrm{info})$ and returns $T_i = h_{H_i}(M_i) \oplus F_i$. The queried nonces $N_1, \ldots, N_q$ must be pairwise distinct.
3. $\mathcal{A}$ outputs (N^*, M^*, T^*).
4. The experiment returns 1 if (N^*, M^*, T^*) is a valid forgery, and returns 0 otherwise.

The EUF-CMA advantage of $\mathcal{A}$ is defined as:

$$\mathrm{Adv}^{\text{EUF-CMA}}_{\mathrm{QMAC}}(\mathcal{A}) = \Pr\left[\mathsf{EUF\text{-}CMA}^{\mathcal{A}}_{\mathrm{QMAC}} = 1\right].$$

A MAC is secure if this advantage is negligible in the security parameter, for all efficient adversaries.

5.3 Universal Hashing and Collision Bounds

The polynomial hash used in QMAC is parameterized by a randomly sampled $H \in GF(2^{256})$. For a message M parsed into blocks $(X_1, \ldots, X_\ell)$, the hash value is:

$$h_H(M) = Y_\ell, \qquad Y_i = H \cdot (Y_{i-1} \oplus X_i), \qquad Y_0 = 0.$$

The collection $\mathcal{H} = \{h_H\}_{H \in GF(2^{256})}$ is treated as a family of hash functions. The standard universal hashing security notions are defined as follows.

The family is ε almost universal (AU) if for all distinct messages $M \neq M'$:

$$\Pr_{H \xleftarrow{\$} GF(2^{256})}[h_H(M) = h_H(M')] \leq \varepsilon.$$

It is ε almost xor universal (AXU) if for all $M \neq M'$ and all $\Delta \in GF(2^{256})$:

$$\Pr_{H \xleftarrow{\$} GF(2^{256})}[h_H(M) \oplus h_H(M') = \Delta] \leq \varepsilon.$$

The polynomial hash over $GF(2^{256})$ instantiated through the irreducible polynomial $m(x)$ satisfies:

$$\varepsilon \leq \ell \cdot 2^{-256},$$

for messages of at most ℓ blocks. These bounds form the universal hashing term of the QMAC unforgeability reduction.

We denote the advantage of an adversary $\mathcal{B}$ in distinguishing collisions from the behavior of an ideal universal hash as:

$$\mathrm{Adv}^{\mathrm{UH}}_{\mathrm{poly}}(\mathcal{B}).$$

5.4 PRF Security of cSHAKE Based Key Derivation

The pairs (H_i, F_i) used by QMAC are derived from the master key by a cSHAKE call per nonce. For the purpose of the reduction, cSHAKE based key derivation is treated as a keyed pseudo-random function of its nonce and customization inputs.

Let KDF_K denote the deterministic keyed derivation function:

$$\mathrm{KDF}_K(N, \mathrm{info}) = (H, F) \in GF(2^{256}) \times GF(2^{256}).$$

We say that cSHAKE based key derivation is a secure PRF if no efficient adversary, given oracle access to either $\mathrm{KDF}_K(\cdot,\cdot)$ for a secret K or to a truly random function with the same domain and range, can distinguish the two. In particular, on the distinct nonces $N_1,\ldots,N_q$ used by a nonce respecting adversary, the real outputs (H_i, F_i) are computationally indistinguishable from q independent uniform pairs in $GF(2^{256}) \times GF(2^{256})$.

The advantage of a PRF adversary $\mathcal{C}$ making at most q queries is defined as:

$$\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{cSHAKE}}(\mathcal{C}) = \left|\Pr[\mathcal{C}^{\mathrm{KDF}_K} \Rightarrow 1] - \Pr[\mathcal{C}^{\$} \Rightarrow 1]\right|,$$

where \$ denotes a random function oracle. This term appears in the final unforgeability bound, combined with the universal hashing advantage.

6 Algebraic Analysis of the Polynomial Hash

The security of QMAC as a Carter Wegman style construction depends on the algebraic properties of its polynomial hash over $GF(2^{256})$. In this section we analyze the collision behavior of the hash family, derive explicit ε almost universal bounds, discuss almost xor universality, and justify the choice of the modulus polynomial.

6.1 Collision Properties over $GF(2^{256})$

Recall that messages are mapped, by the injective padding of Section 2.3, to a block sequence $(X_1,\ldots,X_\ell)$ with $\ell = \lfloor |M|/32 \rfloor + 2$. We first record that this map is injective, which is what rules out the trivial extension and trailing-zero collisions that a naive zero-padding would admit.

Lemma 1 (Injectivity of the padded encoding)**.** *For messages of byte length less than 2^{64}, the map*

$$M \longmapsto \text{Encode-pad}(M) = (X_1,\ldots,X_\ell)$$

is injective. In particular, distinct messages never yield identical block sequences, even when they differ only in length or in trailing zero bytes.

Proof. The length block X_ℓ encodes the exact byte length $|M|$ (as a little-endian 64-bit integer with a fixed `0x01` marker), so two messages with $|M| \neq |M'|$ produce $X_\ell \neq X'_{\ell'}$ and therefore differ. For messages of equal length, $\ell = \ell'$ and the 10^* padding is a fixed injective map on the common-length domain: the position of the `0x80` marker is determined by $|M| \bmod 32$, which is shared, and the preceding bytes are exactly the message bytes. Hence equal-length messages with identical block sequences are equal. As Encode is a bijection on each block, injectivity of the byte-level padding lifts to injectivity of the field-element sequence. □

For a fixed secret key $H \in GF(2^{256})$, the QMAC polynomial hash on the block sequence $(X_1,\ldots,X_\ell)$ is given by

$$Y_0 = 0, \qquad Y_i = H \cdot (Y_{i-1} \oplus X_i), \qquad 1 \le i \le \ell,$$

and we set $h_H(M) = Y_\ell$.

We first express $h_H(M)$ as a polynomial in H with coefficients determined by the message blocks. Expanding the recurrence gives:

$$\begin{aligned}
Y_1 &= H \cdot (0 \oplus X_1) = HX_1,\\
Y_2 &= H \cdot (Y_1 \oplus X_2) = H(HX_1 \oplus X_2) = H^2X_1 \oplus HX_2,\\
Y_3 &= H \cdot (Y_2 \oplus X_3) = H(H^2X_1 \oplus HX_2 \oplus X_3)\\
&= H^3X_1 \oplus H^2X_2 \oplus HX_3,
\end{aligned}$$

and by induction we obtain

$$h_H(M) = Y_\ell = \bigoplus_{i=1}^{\ell} H^{\ell+1-i} X_i.$$

Now let M and M' be two distinct messages, mapping to block sequences $(X_1, \ldots, X_\ell)$ and $(X'_1, \ldots, X'_{\ell'})$ of lengths ℓ and ℓ'. Write $L = \max(\ell, \ell')$ and index both sequences so that the *last* block (the length block) carries the lowest power H^1; concretely, viewing h_H as a polynomial in H,

$$h_H(M) = \bigoplus_{k=1}^{\ell} H^k \, X_{\ell+1-k},$$

so the coefficient of H^k is the k-th block counted from the end. Define the difference coefficients $D_k = X_{\ell+1-k} \oplus X'_{\ell'+1-k}$, where any out-of-range block is taken to be 0. The difference of the hash values is

$$h_H(M) \oplus h_H(M') = \bigoplus_{k=1}^{L} H^k \, D_k =: P(H).$$

We claim P is a nonzero polynomial. If $\ell \neq \ell'$ then $|M| \neq |M'|$, and the coefficient of H^1 equals $X_\ell \oplus X'_{\ell'}$, the difference of the two length blocks, which is nonzero because the length blocks encode different byte lengths. If $\ell = \ell'$ then the length blocks cancel, but by Lemma 1 the remaining block sequences differ, so some $D_k \neq 0$. In both cases $P(H)$ is a nonzero polynomial of degree at most $L \leq \ell_{\max}$, where $\ell_{\max}$ is the larger processed-block count, and it has no constant term.

Since P is a nonzero polynomial of degree at most $\ell_{\max}$ over the field $GF(2^{256})$, it has at most $\ell_{\max}$ roots. Sampling H uniformly from $GF(2^{256})$ (the degenerate weak key $H = 0$, which collapses the hash to 0, occurs with probability 2^{-256} and is absorbed into the bound), it follows that

$$\Pr_{H \xleftarrow{\$} GF(2^{256})}[h_H(M) = h_H(M')] = \Pr[P(H) = 0] \leq \frac{\deg P}{2^{256}} \leq \frac{\ell_{\max}}{2^{256}}.$$

This establishes that the polynomial hash is injective on messages of at most ℓ processed blocks except with probability at most $\ell \cdot 2^{-256}$ over the choice of H, where here and below ℓ denotes the maximum processed-block count of the messages under comparison.

6.2 Epsilon Almost Universal Bounds

We now formalize the almost universality property of the QMAC polynomial hash family. For each $H \in GF(2^{256})$, define the function

$$h_H : \mathcal{M}_{\leq \ell} \to GF(2^{256}),$$

where $\mathcal{M}_{\leq \ell}$ is the set of messages of at most ℓ blocks. The family $\mathcal{H} = \{h_H\}_{H \in GF(2^{256})}$ is said to be $\varepsilon(\ell)$ almost universal if for all distinct messages $M, M' \in \mathcal{M}_{\leq \ell}$,

$$\Pr_{H \xleftarrow{\$} GF(2^{256})}[h_H(M) = h_H(M')] \leq \varepsilon(\ell).$$

From the polynomial argument above, we have established:

Lemma 2. *For any positive integer ℓ and any distinct messages M, M' of at most ℓ blocks, the QMAC polynomial hash satisfies*

$$\Pr_{H \xleftarrow{\$} GF(2^{256})}[h_H(M) = h_H(M')] \leq \frac{\ell}{2^{256}}.$$

In particular, the family $\mathcal{H}$ is $\varepsilon(\ell)$ almost universal with

$$\varepsilon(\ell) \leq \ell \cdot 2^{-256}.$$

In the QMAC construction, H is not sampled directly from $GF(2^{256})$ but derived from the master key via cSHAKE. Under the assumption that the cSHAKE based key derivation is pseudo-random and that H is computationally indistinguishable from a uniform field element, the same $\varepsilon(\ell)$ bound holds from the point of view of any efficient adversary.

6.3 Epsilon Almost Xor Universal Properties

The almost xor universal property considers the distribution of differences of hash values. A hash family $\mathcal{H}$ is ε almost xor universal (AXU) if for all $M \neq M'$ and all $\Delta \in GF(2^{256})$,

$$\Pr_{H \xleftarrow{\$} GF(2^{256})}[h_H(M) \oplus h_H(M') = \Delta] \leq \varepsilon.$$

For the QMAC polynomial hash, using the indexing of Section 6.1 we can write

$$h_H(M) \oplus h_H(M') = \Delta \quad \Longleftrightarrow \quad \bigoplus_{k=1}^{L} H^k \, D_k = \Delta,$$

which is equivalent to the polynomial equation

$$P(H) \oplus \Delta = 0.$$

For fixed M, M', Δ, the left hand side is a polynomial in H of degree at most $\ell_{\max}$, equal to $P(H)$ with its (otherwise zero) constant term shifted by Δ. As long as $M \neq M'$, the difference polynomial P has a nonzero coefficient in some positive degree (by the length-block and injectivity argument of Section 6.1), so $P(H) \oplus \Delta$ is a nonzero polynomial regardless of the choice of Δ. Therefore the same root counting argument gives

$$\Pr_{H \xleftarrow{\$} GF(2^{256})}[h_H(M) \oplus h_H(M') = \Delta] \leq \frac{\ell_{\max}}{2^{256}}.$$

We obtain:

Lemma 3. *For messages of at most ℓ blocks, the QMAC polynomial hash family is $\varepsilon(\ell)$ almost xor universal with*

$$\varepsilon(\ell) \leq \ell \cdot 2^{-256}.$$

In the present work QMAC is used purely as a message authentication code and not as an encryption or xor masking mechanism. The almost universal property is sufficient to derive the unforgeability bounds, but the stronger almost xor universal property also holds and can be relevant for potential composition with other primitives.

6.4 Justification of the Modulus Choice

The field $GF(2^{256})$ used by QMAC is instantiated with the pentanomial

$$m(x) = x^{256} + x^{10} + x^5 + x^2 + 1.$$

This choice is motivated by three considerations.

First, $m(x)$ is irreducible over $GF(2)$, so the quotient $GF(2)[x]/\langle m(x)\rangle$ is a field. This ensures that the polynomial hash is evaluated in an algebraic structure with no zero divisors and that the usual universality arguments apply.

Second, the polynomial has low Hamming weight. Only five nonzero terms are present. This property allows the reduction of a product modulo $m(x)$ to be realized with a small number of xor operations and bit shifts. In the engineering description, this corresponds to the reduction rule

$$x^{256+k} \equiv x^k \oplus x^{k+2} \oplus x^{k+5} \oplus x^{k+10},$$

which can be implemented efficiently in constant time.

Third, the polynomial has been selected to avoid trivial algebraic structure such as repeated factors or easily exploitable linearized forms. In particular, $m(x)$ is not of the form $x^{256} + x^r + 1$ for a reducible trinomial, and it does not introduce a symmetry that would endow the polynomial hash with a hidden period that could be exploited by classical or quantum attacks.

Together, these properties yield a field that supports efficient implementation while preserving the theoretical guarantees of polynomial hashing. The algebraic analysis in this section assumes only irreducibility and field size. Therefore the universality bounds derived above hold for any irreducible polynomial of degree 256, and the specific pentanomial used by QMAC realizes those bounds with an efficient reduction rule.

7 Security Proofs for QMAC

This section presents the main security theorems for QMAC in the classical chosen message setting. The proofs are given as explicit reductions to the universal hashing properties of the polynomial hash and to the pseudo-randomness of the cSHAKE based key derivation. Quantum security considerations are treated separately in the later cryptanalysis sections.

7.1 Universality Lemma for QMAC Polynomial Hashing

We first restate in game oriented form the universality result derived in the algebraic analysis. Informally, if the polynomial hash key H is chosen at random from $GF(2^{256})$, then an adversary cannot force a collision on h_H except with probability proportional to the message length divided by 2^{256}.

Let $\mathcal{M}_{\leq \ell}$ be the set of messages of at most ℓ blocks after parsing and padding, and recall that for $H \in GF(2^{256})$ the polynomial hash h_H is defined by

$$h_H(M) = \bigoplus_{i=1}^{\ell} H^{\ell+1-i} X_i,$$

where $(X_1, \ldots, X_\ell)$ is the encoded block sequence of M and $Y_0 = 0$ in the recurrence form.

Consider an adversary $\mathcal{B}$ that outputs a pair of messages (M, M') with $M \neq M'$. Define the collision experiment:

Experiment $\mathsf{Coll}^{\mathcal{B}}_{\mathrm{poly}}$:

1. Sample $H \xleftarrow{\$} GF(2^{256})$.
2. Run $\mathcal{B}$ to obtain (M, M') with $M \neq M'$.
3. Output 1 if $h_H(M) = h_H(M')$ and 0 otherwise.

The advantage of $\mathcal{B}$ in this experiment is defined as

$$\mathrm{Adv}^{\mathrm{UH}}_{\mathrm{poly}}(\mathcal{B}) = \Pr\left[\mathsf{Coll}^{\mathcal{B}}_{\mathrm{poly}} = 1\right].$$

We obtain the following lemma.

Lemma 4 (Universality of QMAC Polynomial Hash)**.** *Let ℓ be a positive integer. For any probabilistic adversary $\mathcal{B}$ that outputs distinct messages $M, M' \in \mathcal{M}_{\leq \ell}$, the QMAC polynomial hash family satisfies*

$$\mathrm{Adv}^{\mathrm{UH}}_{\mathrm{poly}}(\mathcal{B}) \leq \ell \cdot 2^{-256}.$$

Proof. For fixed distinct M and M', the difference $h_H(M) \oplus h_H(M')$ equals the polynomial $P(H)$ constructed in Section 6.1. By Lemma 1 the padded block sequences of M and M' differ, and by the length-block argument P is a nonzero polynomial in H of degree at most ℓ with no constant term. The equation $h_H(M) = h_H(M')$ is equivalent to $P(H) = 0$.

A nonzero polynomial of degree at most ℓ over a field has at most ℓ roots. Since H is sampled uniformly from $GF(2^{256})$, we have

$$\Pr_{H \xleftarrow{\$} GF(2^{256})}[h_H(M) = h_H(M')] = \Pr[P(H) = 0] \leq \frac{\ell}{2^{256}}.$$

The adversary $\mathcal{B}$ may randomize its output choice of (M, M'), but for each fixed output pair the same bound applies, and the probability is taken over H alone. Therefore

$$\mathrm{Adv}^{\mathrm{UH}}_{\mathrm{poly}}(\mathcal{B}) \leq \ell \cdot 2^{-256},$$

as claimed. □

This lemma gives the universal hashing term that will appear in the final unforgeability bound.

7.2 Idealized Construction and One-Time Analysis

We first analyse an idealized variant, denoted $\mathrm{QMAC}^{\mathsf{UH}}$, in which the key derivation is replaced by truly random sampling. Because QMAC derives a fresh pair (H, F) from each nonce, the idealized variant samples, for each *distinct* nonce N used by the (nonce respecting) adversary, an independent uniform pair

$$(H_N, F_N) \xleftarrow{\$} GF(2^{256}) \times GF(2^{256}),$$

and defines

$$\mathsf{Tag}(N, M) = F_N \oplus h_{H_N}(M).$$

Thus each nonce indexes an independent instance of a one-time Carter–Wegman MAC: the mask F_N is used to authenticate exactly one message, namely the single message queried under nonce N.

Theorem 1 (Unforgeability of the idealized construction)**.** *Let $\mathcal{A}$ be a nonce respecting adversary against QMAC$^{\mathsf{UH}}$ that makes at most q tagging queries, each message having at most ℓ processed blocks, and outputs a single forgery attempt (N^*, M^*, T^*). Then*

$$\mathrm{Adv}^{\text{EUF-CMA}}_{\text{QMAC}^{\mathsf{UH}}}(\mathcal{A}) \leq \ell \cdot 2^{-256}.$$

More generally, an adversary permitted q_v forgery (verification) attempts succeeds with probability at most $q_v \cdot \ell \cdot 2^{-256}$.

Proof. Consider the forgery attempt (N^*, M^*, T^*) and condition on the adversary's view, which consists of its queries (N_i, M_i) and the returned tags $T_i = F_{N_i} \oplus h_{H_{N_i}}(M_i)$, with the nonces N_i pairwise distinct. There are two cases.

Case 1: $N^* \notin \{N_1, \ldots, N_q\}$. Then the pair (H_{N^*}, F_{N^*}) is sampled independently of every pair used to answer the queries and is independent of the adversary's entire view. In particular F_{N^*} is uniform and independent of $\mathcal{A}$'s view, so the valid tag $T_{N^*}(M^*) = h_{H_{N^*}}(M^*) \oplus F_{N^*}$ is uniformly distributed and independent of T^*. Hence

$$\Pr[T^* = T_{N^*}(M^*)] = 2^{-256}.$$

Case 2: $N^* = N_j$ for the unique query index j with that nonce. Validity requires $M^* \neq M_j$ (otherwise the forgery replays a queried pair and is disallowed). The adversary has observed exactly one tag under the pair (H_{N_j}, F_{N_j}), namely $T_j = h_{H_{N_j}}(M_j) \oplus F_{N_j}$. For each candidate value of H_{N_j} there is exactly one mask F_{N_j} consistent with the observed T_j; since F_{N_j} was uniform and independent of H_{N_j}, the posterior distribution of H_{N_j} given the view is still uniform on $GF(2^{256})$. A correct forgery requires

$$T^* = h_{H_{N_j}}(M^*) \oplus F_{N_j} = T_j \oplus \big(h_{H_{N_j}}(M^*) \oplus h_{H_{N_j}}(M_j)\big),$$

i.e. $\mathcal{A}$ must predict the value $\delta = h_{H_{N_j}}(M^*) \oplus h_{H_{N_j}}(M_j) = T^* \oplus T_j$. By the almost xor universal property (Lemma 3), for the fixed distinct messages M^*, M_j and the value δ determined by $\mathcal{A}$'s output, over the uniform posterior of H_{N_j},

$$\Pr_{H_{N_j}}\Big[h_{H_{N_j}}(M^*) \oplus h_{H_{N_j}}(M_j) = \delta\Big] \leq \ell \cdot 2^{-256}.$$

Taking the maximum over the two cases, a single forgery attempt succeeds with probability at most $\ell \cdot 2^{-256}$. The bound for q_v attempts follows by a union bound over the attempts. □

Two features of this argument are worth emphasizing. First, the bound does *not* grow with the number of tagging queries q: because every query uses a distinct nonce, the masks F_{N_i} are genuine one-time pads and the forgery can interact algebraically with at most one of them. This is tighter than the bound of the form $q \cdot \ell \cdot 2^{-256}$ that arises in settings where a single mask is reused. Second, the argument relies essentially on each mask being used only once; if two messages were ever tagged under the same (H_N, F_N), the one-time-pad property of F_N would fail and the analysis would not apply. This is exactly the nonce-reuse condition treated in Section 7.6.

7.3 Reduction from EUF-CMA to cSHAKE PRF Security

We next relate real QMAC, which derives each (H_i, F_i) from KDF_K, to the idealized QMAC$^{\mathsf{UH}}$ of Theorem 1, in which the per-nonce pairs are independent and uniform. This is a single PRF game hop on the keyed derivation function.

Game 0 (Real QMAC). The EUF-CMA experiment of Section 5, where each query under nonce N_i is answered with $(H_i, F_i) = \mathrm{KDF}_K(N_i, \text{info})$ and the forgery is verified with $\mathrm{KDF}_K(N^*, \text{info})$.

Game 1 (Random function). The same experiment, except that KDF_K is replaced by a truly random function $\mathcal{R}$ from the nonce/customization domain to $GF(2^{256}) \times GF(2^{256})$. Because the queried nonces are distinct and the forgery uses a single nonce, the pairs supplied in Game 1 are exactly the independent uniform pairs of $\mathrm{QMAC}^{\mathsf{UH}}$.

By definition,

$$\mathrm{Adv}^{\text{EUF-CMA}}_{\mathrm{QMAC}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ forges in Game 0}],$$
$$\mathrm{Adv}^{\text{EUF-CMA}}_{\mathrm{QMAC}^{\mathsf{UH}}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ forges in Game 1}].$$

We build a PRF distinguisher $\mathcal{C}$ against KDF_K. Given an oracle $\mathcal{O}$ that is either KDF_K or a random function $\mathcal{R}$, the distinguisher $\mathcal{C}$ runs $\mathcal{A}$, answering each tagging query under nonce N_i by querying $\mathcal{O}(N_i, \text{info})$ to obtain (H_i, F_i) and returning $h_{H_i}(M_i) \oplus F_i$, and verifying the final forgery by one more query $\mathcal{O}(N^*, \text{info})$. If $\mathcal{A}$'s output is a valid forgery, $\mathcal{C}$ outputs 1, else 0. Then $\mathcal{C}$ makes at most $q + 1$ queries to $\mathcal{O}$ (all on distinct inputs whenever N^* is fresh, and otherwise reusing a value already obtained), and

$$\Pr[\mathcal{C}^{\mathrm{KDF}_K} \Rightarrow 1] = \Pr[\mathcal{A} \text{ forges in Game 0}],$$
$$\Pr[\mathcal{C}^{\mathcal{R}} \Rightarrow 1] = \Pr[\mathcal{A} \text{ forges in Game 1}].$$

Hence

$$\left|\mathrm{Adv}^{\text{EUF-CMA}}_{\mathrm{QMAC}}(\mathcal{A}) - \mathrm{Adv}^{\text{EUF-CMA}}_{\mathrm{QMAC}^{\mathsf{UH}}}(\mathcal{A})\right| = \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{cSHAKE}}(\mathcal{C}),$$

and therefore

$$\mathrm{Adv}^{\text{EUF-CMA}}_{\mathrm{QMAC}}(\mathcal{A}) \le \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{cSHAKE}}(\mathcal{C}) + \mathrm{Adv}^{\text{EUF-CMA}}_{\mathrm{QMAC}^{\mathsf{UH}}}(\mathcal{A}).$$

7.4 Combined Security Bound

We now combine Theorem 1 with the PRF game hop to obtain the main result.

Theorem 2 (EUF-CMA security of QMAC)**.** *Let $\mathcal{A}$ be a nonce respecting adversary against QMAC that makes at most q tagging queries (on pairwise distinct nonces), each message having at most ℓ processed blocks, and at most q_v forgery attempts. Then there is a PRF distinguisher $\mathcal{C}$ against KDF_K, making at most $q + q_v$ queries and running in essentially the same time as $\mathcal{A}$ plus the cost of simulating the oracle, such that*

$$\mathrm{Adv}^{\text{EUF-CMA}}_{\mathrm{QMAC}}(\mathcal{A}) \le \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{cSHAKE}}(\mathcal{C}) + q_v \cdot \ell \cdot 2^{-256}.$$

In particular, for a single forgery attempt ($q_v = 1$),

$$\mathrm{Adv}^{\text{EUF-CMA}}_{\mathrm{QMAC}}(\mathcal{A}) \le \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{cSHAKE}}(\mathcal{C}) + \ell \cdot 2^{-256}.$$

Proof. Immediate from the PRF game hop of Section 7.3 and Theorem 1. □

This is the main security theorem for QMAC in the classical chosen message setting. The number of tagging queries q enters the bound only through the PRF advantage of the key derivation, not through the universal hashing term; the latter depends on the number of forgery attempts and the message length, but not on q, because distinct nonces yield independent one-time masks.

7.5 Discussion of Tightness and Parameter Choices

The bound obtained above is of the standard Carter Wegman form. It consists of a term that reflects the pseudo-randomness of the key derivation and a term that reflects the universality of the hash family. The universal hashing term grows linearly in the maximal number of blocks processed per key and is inversely proportional to the field size.

In practice, with a 256 bit field, the universal hashing term $q_v \cdot \ell \cdot 2^{-256}$ remains negligible for any realistic number of forgery attempts and message length. For example, for $q_v \cdot \ell \leq 2^{64}$, the universal hashing term is at most 2^{-192}. The dominant factor in the security level is therefore the PRF security of cSHAKE and the effective resistance of the construction to quantum attacks, both of which are discussed in later sections.

The reduction is essentially tight with respect to the universal hashing component: generic attacks that search for collisions in polynomial hashing over $GF(2^{256})$ can achieve advantage comparable to the bound when the number of queries approaches the square root of the field size. The PRF reduction to cSHAKE is also tight in the sense that an adversary that can distinguish (H, F) from uniform can be directly used to break QMAC.

These considerations guide parameter choices and usage limits. The 256 bit tag length provides ample margin for both classical and quantum adversaries, provided that keys and nonces are managed correctly and that the total amount of authenticated data per key remains well below the regime where the universal hashing term becomes non negligible. The subsequent cryptanalytic sections refine these conclusions in the Q1 and Q2 models and in comparison with related constructions such as GMAC, Poly1305, and KMAC.

7.6 Security under Nonce Reuse

The unforgeability theorem assumes a nonce respecting adversary. We now make explicit why this assumption is necessary by describing the attack that becomes available when two messages are authenticated under the same nonce, and hence under the same derived pair (H, F). This is the QMAC analogue of the well known forbidden-attack on polynomial MACs of the GMAC and GCM family.

Suppose an implementation reuses a nonce N for two distinct messages M and M', producing tags

$$T = h_H(M) \oplus F, \qquad T' = h_H(M') \oplus F.$$

The mask cancels in the difference:

$$T \oplus T' = h_H(M) \oplus h_H(M') = P(H),$$

where P is the nonzero difference polynomial of Section 6.1, of degree at most ℓ, with coefficients that the adversary can compute from the (known) message blocks. The adversary therefore obtains a polynomial equation $P(H) \oplus (T \oplus T') = 0$ that the secret H satisfies. Factoring this degree-ℓ polynomial over $GF(2^{256})$ is a polynomial-time computation and yields at most ℓ candidate roots; additional nonce-colliding message pairs intersect the candidate sets and isolate the true H. Once H is known, $F = T \oplus h_H(M)$ follows, and the adversary can forge a valid tag for *any* message under that nonce. Recovery thus requires only a small number of nonce-colliding tags and polynomial work, in sharp contrast to the 2^{128}-level effort against a correctly used instance.

Remark 1. The damage from a repeated nonce is confined to the key pair (H_N, F_N) associated with that nonce: because QMAC re-derives (H, F) from each nonce, recovering H_N does not reveal the master key K nor compromise tags produced under other nonces, provided the cSHAKE based derivation is a secure PRF. Nevertheless, the consequence for the affected nonce is total (universal forgery), so unique nonces per message under a fixed key are mandatory. Section 10.3 states the corresponding implementation requirements.

We now analyze the security of QMAC in the presence of quantum adversaries. The discussion is informal in the sense that we do not give full quantum reductions, but we align the construction with known results on polynomial MACs and hash based primitives in quantum oracle models. The main goals are to clarify which quantum models are considered, how they affect the security terms in the classical bound, and why QMAC does not appear to instantiate the hidden-period structure exploited by known Simon-style attacks against some polynomial MACs and AE modes.

7.7 Quantum Adversaries and Oracle Models

In the quantum setting we distinguish between two modes of oracle access.

- In the *Q1* model, the adversary has only classical access to the MAC oracle, but may use internal quantum computation to process the information it receives.
- In the *Q2* model, the adversary is allowed to query the MAC oracle in quantum superposition. The oracle is modeled as a unitary that maps

$$\sum_M \alpha_M |M\rangle|0\rangle \quad \longmapsto \quad \sum_M \alpha_M |M\rangle|T(M)\rangle,$$

  where $T(M)$ is the QMAC tag for message M.

The Q1 model captures settings where the MAC interface is classical but the adversary can use quantum resources internally. The Q2 model is stronger; it assumes that the tagging functionality itself is accessible to quantum queries, which is more conservative but appropriate for theoretical analysis.

For cSHAKE and the underlying Keccak permutation, we adopt a quantum oracle perspective similar to the quantum random oracle model. The cSHAKE based key derivation is treated as a pseudo-random function that remains hard to distinguish from a random function even when the adversary can issue quantum queries. This quantum PRF assumption is stronger than the classical one, but is consistent with prevailing analyses of Keccak based constructions.

7.8 Quantum Bounds for Polynomial MACs

The classical security proofs for QMAC rely on the almost universal and almost xor universal properties of the polynomial hash family. Quantum adversaries can in principle obtain more information from oracle queries, so the effective collision and forgery bounds may degrade compared to the classical case.

There are two main generic quantum effects to consider.

- Grover style search reduces the complexity of exhaustive key search or tag guessing from 2^n to roughly $2^{n/2}$ operations.
- Quantum collision finding algorithms can reduce the complexity of finding collisions in random functions from $2^{n/2}$ to roughly $2^{n/3}$ queries in certain settings.

For polynomial MACs of Carter Wegman type, prior work shows that quantum adversaries do not obtain arbitrary structural shortcuts simply from the linearity of the hash. The universal hashing term in the unforgeability bound can degrade by at most a polynomial factor in the number of queries, and the overall picture remains that a field of size 2^n provides roughly n bits of security in the classical setting and somewhat less in the quantum setting, depending on the exact oracle model and attack.

A key point is that the polynomial hash used in QMAC does not define a function with a global xor period. For a fixed key H, the map $M \mapsto h_H(M)$ does not satisfy a Simon type promise of the form

$$\exists s \neq 0 \text{ such that } h_H(M) = h_H(M \oplus s) \quad \text{for all } M,$$

nor does the full tag function $M \mapsto T(M) = h_H(M) \oplus F$ exhibit a two to one xor periodic structure. Any such global period would contradict the almost universal properties established in the algebraic analysis. Consequently, QMAC does not appear to instantiate the direct hidden-period promise exploited by Simon-style period-finding attacks. This observation is not a full Q2 security proof; it only separates QMAC from the specific global-period structures used in known attacks.

In summary, QMAC inherits the known quantum bounds of polynomial MACs where the effective advantage of an adversary that makes q quantum queries to the MAC oracle grows faster than in the classical case, but without any known structural attack that would invalidate the construction.

7.9 Quantum Security of cSHAKE Key Derivation

QMAC invokes cSHAKE once per MAC initialization, deriving one per-nonce pair (H_N, F_N) from the master key, nonce, and information string. A master key may authenticate many messages, provided each message uses a distinct nonce. An adversary that has oracle access to QMAC does not obtain direct oracle access to the internal cSHAKE computation. As a result, the effective exposure of cSHAKE in QMAC is mediated through polynomial tags rather than through raw cSHAKE outputs or a direct cSHAKE MAC interface.

The PRF term in the classical unforgeability bound,

$$\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{cSHAKE}}(\mathcal{C}),$$

must be interpreted in a quantum setting as the advantage of a quantum adversary in distinguishing the derived subkeys (H, F) from independent uniform field elements. Since QMAC uses one KDF call per nonce and does not reveal the derived outputs (H_N, F_N) directly, the adversary cannot query cSHAKE as a stand-alone oracle through the QMAC interface; it sees only the effect of the derived pair through the polynomial tag equation for each nonce.

We assume that cSHAKE instantiated with appropriate parameters satisfies a quantum PRF security notion, so that even in the presence of quantum computations and Q2 style access to the MAC oracle, the distribution of (H, F) remains computationally indistinguishable from uniform to any efficient adversary. Under this assumption, the PRF term in the QMAC bound continues to behave like a negligible term comparable to the classical setting.

7.10 Resulting Quantum Security Levels

Combining these observations with the classical reduction in Section 6, we obtain the following qualitative picture for quantum security.

- The universal hashing term $q_v \cdot \ell \cdot 2^{-256}$ remains the baseline collision related component. In the presence of quantum queries, one expects effective bounds of comparable form but with potential polynomial losses arising from quantum collision finding algorithms. Since 2^{256} is large, the resulting bounds remain extremely small for realistic parameter choices.
- The PRF term for cSHAKE remains the dominant concern for long term quantum security. Under standard quantum PRF assumptions for Keccak based constructions, the effective security level against key recovery and forgery remains close to 128 bits in a conservative interpretation, due to generic Grover style search against the 256 bit key space.
- No direct Simon-style Q2 attack is known for QMAC. The tag oracle does not appear to implement a two-to-one xor-periodic function with a hidden shift, and a global period would be inconsistent with the almost-universality properties proved for the polynomial hash. A complete Q2 proof remains outside the scope of this paper.

In the Q1 model, treating QMAC as providing roughly 128 bits of post-quantum brute-force resistance is conservative and aligns with standard interpretations of 256-bit symmetric primitives under Grover-style search. In the Q2 model, this paper does not claim a complete superposition-query proof; it records only that no direct hidden-period attack

is known for the QMAC tag function. This level is sufficient for long term confidentiality and integrity in many applications. The absence of known structure specific quantum attacks, combined with the Carter Wegman form of the construction and the strong field size, suggests that QMAC remains robust in a quantum setting, provided that keys and nonces are managed correctly and that the total volume of authenticated data per key stays within conservative bounds.

8 Cryptanalytic Analysis and Comparative Evaluation

This section provides a qualitative cryptanalytic assessment of QMAC and compares it with related polynomial and sponge based MAC constructions. The focus is on diffusion, linear mixing, structural vulnerabilities, and work factor estimates in both classical and quantum settings.

8.1 Diffusion and Linear Mixing Properties

QMAC uses multiplication in $GF(2^{256})$ defined by the irreducible pentanomial

$$m(x) = x^{256} + x^{10} + x^5 + x^2 + 1.$$

Each message block update applies the transformation

$$Y \leftarrow H \cdot (Y \oplus X_i),$$

where H is the fixed secret hash key, Y is the current accumulator, and X_i is the encoded block. This is an affine map over $GF(2^{256})$ of the form

$$Y \mapsto H \cdot Y \oplus H \cdot X_i.$$

For a fixed nonzero H, multiplication by H in $GF(2^{256})$ is a linear permutation on the 256 dimensional vector space over $GF(2)$. The associated linear map has no nontrivial kernel, so no component of Y can be fixed or annihilated by the update. The choice of a low weight irreducible polynomial ensures that the reduction step after polynomial multiplication distributes individual bit contributions across many output coordinates. In terms of diffusion, a single input bit of Y or X_i influences multiple output bits after one multiplication, and the number of affected bits grows quickly with successive rounds.

Compared with GMAC, which operates in $GF(2^{128})$, QMAC doubles the field dimension. This yields two benefits. First, the linear map induced by multiplication by H acts on a larger state space, which raises the cost of any linear algebra based attack that attempts to recover H from observed tags and chosen messages. Second, the collision probability of the polynomial hash decreases from an order of magnitude of 2^{-128} per block to 2^{-256} per block, as formalized in the algebraic analysis.

From a structural perspective, the linearity of the polynomial hash is intentional and well understood. It does not introduce a global period or an easily exploitable invariant. For a fixed key H, the map $M \mapsto h_H(M)$ is injective except with the small probability quantified in Lemma 4. In particular, there is no nonzero mask Δ such that

$$h_H(M) = h_H(M \oplus \Delta) \quad \text{for all } M,$$

and hence the full tag function $M \mapsto T(M) = h_H(M) \oplus F$ does not appear to satisfy a Simon-type promise of the form required by known hidden-period attacks. Any global xor period of that form would be inconsistent with the almost universal and almost xor universal properties established earlier. This absence of an obvious structural period is relevant in the Q2 model and is revisited in the quantum security discussion.

8.2 Comparison with GMAC, Poly1305, and KMAC

QMAC is structurally closest to GMAC and Poly1305. All three constructions use algebraic message authentication mechanisms built around polynomial evaluation or polynomial multiplication, but they differ in field or ring size, key derivation, masking method, tag length, and quantum-model posture. GMAC operates over $GF(2^{128})$ and is normally used with a 128-bit authentication tag. Poly1305 evaluates a polynomial modulo $2^{130} - 5$ and is also normally used with a 128-bit tag in ChaCha20-Poly1305. QMAC evaluates a polynomial hash over $GF(2^{256})$, derives a fresh per-nonce hash key and finalization mask from cSHAKE, and emits a 256-bit tag.

Tag length and collision bounds. GMAC's universal-hashing term is in the 128-bit field regime, and Poly1305's authenticator likewise targets the compact 128-bit authentication regime used by its surrounding AEAD construction. QMAC uses a 256-bit field and 256-bit tag, yielding the nonce-respecting universal-hashing term $\ell \cdot 2^{-256}$ for messages of at most ℓ processed blocks. This gives a larger classical universal-hashing margin and a more conservative generic quantum-search margin than 128-bit polynomial authenticators, provided nonces are unique.

KMAC is not a polynomial MAC. It derives its security primarily from the PRF behavior of the keyed Keccak sponge and can use variable output lengths. Its proof structure is therefore different from QMAC's Carter-Wegman proof. QMAC exposes more algebraic structure than KMAC, but that structure also gives a direct root-counting proof for the nonce-respecting universal-hashing term.

Performance and message size regimes. In GMAC and Poly1305, authentication cost is dominated by polynomial multiplication in the selected field or ring. QMAC has the same broad computational shape but uses carryless multiplication and reduction over $GF(2^{256})$. For typical message sizes, QMAC behaves like a fixed number of field multiplications per block, which is amenable to vectorization and may be attractive on platforms where carryless multiplication instructions are efficient or where a Keccak implementation is already present for key derivation.

KMAC evaluates the keyed sponge over the entire message. Its performance depends on the Keccak rate and permutation cost and is usually favorable for long messages. QMAC's advantage, when present, comes from separating the per-message cSHAKE derivation from a regular polynomial block-processing loop.

Implementation complexity and code footprint. GMAC requires a block cipher, a block-cipher key schedule, and $GF(2^{128})$ multiplication. Poly1305 requires arithmetic modulo $2^{130} - 5$ and a surrounding mechanism for deriving a one-time polynomial key and mask. QMAC requires cSHAKE plus a small amount of carryless field arithmetic over $GF(2^{256})$. KMAC requires only the Keccak-derived functions and no explicit polynomial arithmetic. The appropriate choice therefore depends on whether a deployment values direct sponge authentication, compact tags, polynomial throughput, or larger-field nonce-respecting authentication margins.

8.3 Work Factor Estimates and Usage Guidelines

The classical unforgeability bound for QMAC has the form

$$\mathrm{Adv}^{\text{EUF-CMA}}_{\text{QMAC}}(\mathcal{A}) \leq \mathrm{Adv}^{\text{PRF}}_{\text{cSHAKE}}(\mathcal{C}) + q_v \cdot \ell \cdot 2^{-256},$$

where q_v is the number of forgery (verification) attempts and ℓ is an upper bound on the number of processed blocks per message. The first term reflects the difficulty of

distinguishing the derived (H, F) pairs from uniform, and grows with the number of tagging queries q only through the PRF advantage. The second term reflects the universal hashing contribution and is independent of q in the nonce respecting setting.

From a work factor perspective, an adversary that simply guesses tags has success probability of 2^{-256} per attempt. Even large scale brute force forgery attempts remain infeasible at this level. The universal hashing term becomes significant only when $q_v \cdot \ell$ approaches the order of 2^{256}, which is far beyond plausible usage levels.

In the presence of quantum adversaries, Grover style search suggests that exhaustive search against a 256 bit key space costs on the order of 2^{128} quantum operations. Quantum collision or forgery strategies against polynomial MACs may see a similar reduction in effective security, but there is no indication of a structural attack that would push the complexity below this range. Treating QMAC as providing approximately 128 bits of security against quantum adversaries is conservative and consistent with standard symmetric cryptography assessments for 256 bit fields and tags.

These considerations lead to practical usage guidelines:

- A unique nonce must be used for every message under a fixed master key. Nonce reuse enables the polynomial key-recovery attack of Section 7.6 and leads to universal forgery for the affected nonce.
- Keys and nonces should never be reused across independent cryptographic domains. Domain separation in the cSHAKE invocation must be maintained.
- The number of processed blocks per message ℓ and the number of forgery attempts q_v should remain well below the regime where $q_v \cdot \ell$ approaches 2^{128}, which already provides a very generous margin in both classical and quantum models.
- Implementations should ensure that the polynomial arithmetic and cSHAKE invocations are constant time with respect to secret data to avoid side channel leaks that are outside the formal model.

Within these constraints, QMAC provides strong nonce-respecting integrity guarantees. Its field size and tag length give a larger universal-hashing margin than GMAC and Poly1305, while its use of cSHAKE makes it natural in environments where Keccak based primitives are already deployed.

9 Implementation Conformance and Side Channel Considerations

This section identifies the implementation invariants required by the formal QMAC construction and records how the reference implementation realizes them. The objective is to connect the mathematical model to concrete state transitions, constant-time arithmetic requirements, and deployment assumptions.

9.1 Mapping Between Model and Reference Implementation

The reference C code implements QMAC in direct accordance with the formal construction. Each component of the mathematical specification has a corresponding concrete operation.

Key derivation. The formal key derivation step

$$(H, F) = \mathrm{Split}_{256,256}(\mathrm{cSHAKE}(K, N, \mathrm{info}))$$

is realized in the function `qsc_qmac_initialize`. The implementation performs a single cSHAKE absorption, with the master key as the keyed input and the nonce and information

string as the function-name and customization inputs, then squeezes one output block and uses its first 64 bytes, taking the first 32 bytes as H and the next 32 bytes as F. No additional processing of the cSHAKE output occurs, which matches the formal model exactly.

Block encoding and padding. Messages are divided into 32 byte blocks. The trailing block is processed by `qmac_compute_final`, which writes a `0x80` marker byte after the buffered message bytes and zero-fills the rest of the block (`10*` padding), then absorbs a separate length block: `qsc_intutils_le64to8` writes the processed byte length into the low eight bytes of `lbuf` and `lbuf[31]` is set to `0x01`. The function `qsc_qmac_update` copies each full 32-byte block into a four-word array using the implementation's fixed little-endian word ordering. The encoding therefore coincides with the injective Encode-pad map of the formal description, including the length block.

Accumulator update. The accumulator update

$$Y \leftarrow H \cdot (Y \oplus X_i)$$

matches the exact sequence in `qmac_block_update`. The xor with the message block is performed first, followed by a field multiplication using `qmac_gfmul256_poly`, which implements multiplication modulo the specified pentanomial. This matches the defined recurrence exactly.

Finalization. The formal finalization step

$$T = Y_\ell \oplus F$$

is implemented in `qmac_compute_final`. After the padded trailing block and the length block have been absorbed by `qmac_process_bytes`, the accumulator is xor combined with the finalization key F and copied to the output buffer. No additional hashing or mixing occurs after the combination with F.

Field arithmetic. The reference multiplication routine `qmac_gfmul256_poly` computes a 512-bit carryless product (either via the portable constant-time routine `qmac_clmul256_ref` or, where available, a CLMUL-based path) and reduces it with `qmac_reduce256` using the relation

$$x^{256} \equiv x^{10} \oplus x^5 \oplus x^2 \oplus 1.$$

The reduction folds the high 256 bits back into the low 256 by xoring shifted copies at offsets $0, 2, 5, 10$; the bits shifted beyond position 255 are collected into a small overflow word and folded a second time at the same offsets, which is exact because the overflow has degree below 10 and the second fold cannot itself overflow. This two-stage fold matches the reduction rule of the formal model, and the mapping between the polynomial and bit-string representations is consistent with the formal encoding.

Together, these correspondences confirm that the implementation is a faithful realization of the formal QMAC construction.

9.2 Constant Time Behavior and Microarchitectural Issues

The implementation is structured to avoid data dependent branches and memory accesses, which is essential for preventing timing leakage and side channel vulnerabilities.

Constant time field multiplication. The core operation `qmac_gfmul256_poly` performs carryless multiplication and modular reduction using loops that operate over fixed index ranges. The decisions made during reduction depend only on arithmetic on public bit positions, not on secret dependent control flow. All bit tests, shifts, and xors are unconditional.

Fixed memory access patterns. All loads and stores during block processing operate on fixed sized buffers, and there are no secret indexed table lookups. Each message block is copied into a fixed array of four 64 bit words, and all subsequent arithmetic uses registers or contiguous memory operations.

Vectorized operations. The implementation uses vectorized xor and copy operations when available. These operate on fixed width lanes and do not introduce data dependent branching. Scalar fallbacks exist and behave deterministically with respect to secret data.

Code paths independent of secret values. The implementation does not vary control flow based on the values of H, F, Y, or block contents. All high level code paths are determined solely by message length and not by secret information.

These properties ensure that the implementation conforms to the constant time assumptions required by the security model. The lack of table lookups or branch dependent arithmetic protects against timing side channels, cache attacks, and microarchitectural leakage.

9.3 Randomness and Nonce Handling

The formal model requires that each QMAC computation use a nonce that is unique for each master key. The implementation treats the nonce as caller supplied. This design requires the surrounding system to enforce the following conditions.

- The nonce must be unique for each initialization under a fixed master key. Reuse of (K, N) pairs regenerates the same (H, F) pair; authenticating two messages under it enables the polynomial key-recovery attack of Section 7.6 and results in universal forgery for that nonce.
- The nonce must be exactly 32 bytes and must not be truncated or padded inconsistently.
- The implementation does not verify the quality of randomness in the nonce. Therefore the system must ensure either uniformly random or at minimum unique nonce generation.
- The master key K is not reused across domains with different cSHAKE customizations. Domain separation must be applied externally if QMAC is used in multiple protocol components.

These requirements match the assumptions in the security model of Section 5, where the adversary is nonce respecting and the nonce N is treated as an adversarially observable but externally controlled value that must be unique per message.

9.4 Key and State Erasure

The formal security model assumes that ephemeral state and secret keys are erased after use. The implementation includes explicit clearing of all sensitive material.

Internal state clearing. The function `qsc_qmac_dispose` clears H, F, and Y using secure clearing primitives that overwrite memory before releasing control. This satisfies the erasure requirement for the hash subkey, finalization key, and accumulator.

Temporary buffers and Keccak state. The cSHAKE initialization routine uses temporary buffers to hold intermediate data. These buffers and the Keccak state are cleared immediately after (H, F) are extracted. No residues of the internal sponge state remain in memory after initialization.

No persistent state. QMAC does not store any state between calls other than what is explicitly passed in the state structure. This supports the assumption that the MAC computation leaves no covert state for the adversary to exploit across sessions.

The erasure behavior of the implementation satisfies the assumptions of the formal model and contributes to side channel resilience by preventing leakage of stale secret data.

10 Conclusion

10.1 Summary of Results

This paper presented a formal analysis of QMAC, a Carter Wegman style message authentication code that combines a polynomial hash over $GF(2^{256})$ with subkeys derived through cSHAKE. We established a clean mathematical model for the construction, provided an implementation-aligned engineering level description grounded in the reference implementation, and derived explicit unforgeability bounds in the chosen message setting.

The algebraic analysis demonstrated that the polynomial hash, evaluated over the injective 10^*-plus-length encoding, satisfies strong almost universal and almost xor universal properties, with collision probability bounded by $\ell \cdot 2^{-256}$ for messages of at most ℓ processed blocks. Treating QMAC as a nonce based MAC, in which each message is processed under an independently derived pair (H, F), we proved that any nonce respecting EUF-CMA adversary reduces to a distinguisher against the cSHAKE based key derivation together with the one-time universal hashing bound, yielding $\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{cSHAKE}} + q_v \cdot \ell \cdot 2^{-256}$. We also made explicit the algebraic key-recovery attack that applies under nonce reuse, delimiting precisely the regime in which the proof holds. The combined bound takes the classical Carter Wegman form and yields a negligible forgery probability for any realistic usage.

The quantum analysis argued that QMAC does not appear to expose the global hidden-period structure exploited by known Simon-style attacks. In the Q1 model, quantum adversaries primarily gain generic advantages such as Grover-style search speedup, and the effective brute-force security level is comparable to that of other 256-bit symmetric constructions. A complete Q2 proof is not claimed. The cSHAKE based key derivation remains the dominant factor in quantum settings, and the polynomial hash inherits known quantum bounds for universal hash based MACs.

The cryptanalytic comparison placed QMAC alongside GMAC, Poly1305, and KMAC in terms of algebraic structure, field size or sponge capacity, tag length, implementation complexity, and work factors. The larger field and tag size give QMAC a significantly larger nonce-respecting universal-hashing margin than compact 128-bit polynomial authenticators, while its structure remains simple enough for direct algebraic analysis. The implementation conformance review confirmed that the reference code aligns with the formal model, operates without data dependent branching, and incorporates secure key erasure.

10.2 Limitations and Future Work

The analysis presented here follows the standard Carter Wegman framework and assumes that cSHAKE behaves as a pseudo-random function in both classical and quantum oracle models. While this assumption is consistent with existing studies of Keccak based primitives, further work on tighter quantum reductions for sponge based KDFs would strengthen the theoretical foundations of QMAC in the Q2 setting.

Another area for future study is the exploration of alternative irreducible polynomials or field dimensions. The present construction uses a degree 256 pentanomial for its balance of efficiency and algebraic clarity. Other choices may offer alternative tradeoffs between performance and hardware acceleration opportunities. Formal analysis of such variants would follow similar lines but may require adjustments to implementation strategies.

A further consideration concerns behaviour under nonce misuse. As shown in Section 7.6, QMAC is *not* misuse resistant: a single repeated nonce permits algebraic recovery of the per-nonce hash key and consequent universal forgery for that nonce, in line with other polynomial MACs of the GMAC family. The construction therefore depends critically on unique nonces. A worthwhile direction for future work is a misuse resistant variant, for example by deriving the finalization mask from a synthetic value that also depends on the message, so that accidental nonce repetition degrades gracefully rather than catastrophically.

10.3 Implications for Deployment

QMAC is well suited for deployment in systems that require a symmetric integrity mechanism with strong long term security guarantees and compatibility with post quantum design goals. Its reliance on cSHAKE for key derivation fits naturally into protocol stacks that already include Keccak based components. The implementation is compact, predictable, and straightforward to verify, which supports use in constrained environments.

The security margin provided by the 256 bit field and tag length is substantial. Even in the presence of quantum adversaries, the expected work factor for successful forgery remains on the order of 2^{128} operations. When combined with careful nonce management, constant time implementation practices, and proper key erasure, QMAC provides a robust and analytically grounded MAC suitable for modern cryptographic deployments.

Overall, QMAC offers a blend of simplicity, strong nonce-respecting theoretical guarantees, and implementation friendliness that make it a viable choice for symmetric authentication in systems that can enforce unique nonces. Future refinements in quantum analysis and implementation optimization will further enhance its suitability for long term secure applications.

11 References

1. National Institute of Standards and Technology (NIST). *FIPS 202: SHA-3 Standard.* U.S. Department of Commerce, 2015. Available at: https://doi.org/10.6028/NIST.FIPS.202.

2. National Institute of Standards and Technology (NIST). *SP 800-185: SHA-3 Derived Functions.* U.S. Department of Commerce, 2016. Available at: https://doi.org/10.6028/NIST.SP.800-185.

3. Carter, J. L., and Wegman, M. N. *Universal Classes of Hash Functions.* Journal of Computer and System Sciences, 1979. Available at: https://doi.org/10.1016/0022-0000(79)90044-8.

4. Boneh, D., and Zhandry, M. *Secure Message Authentication Codes in the Quantum Setting.* Eurocrypt 2013. Available at: https://doi.org/10.1007/978-3-642-38348-9_29.

5. Underhill, J. G. *QMAC Technical Specification.* Quantum Resistant Cryptographic Solutions Corporation, 2025. Available at: https://www.qrcscorp.ca/documents/qmac_specification.pdf

6. QRCS Corporation. *The QSC Cryptographic Library.* Quantum Resistant Cryptographic Solutions Corporation, 2025. Available at: https://github.com/QRCS-CORP/QSC